Documentação Classificador baseado em SVM treinado com a família Amadey
Projeto de finalização da cadeira de criação de antivírus (Pós-graduação em Defesa Cibernética, Blue Team)

Objetivo do Projeto

Treinar um classificador SVM (Support Vector Machine) capaz de distinguir arquivos executáveis PE maliciosos (família de malware Amadey) de arquivos benignos, a partir de características extraídas por análise estática (engenharia reversa), utilizando o template de antivírus SVM do repositório DejavuForensics/SVM.

Visão Geral do Pipeline

1. Coleta de amostras maliciosas da família Amadey (MalwareBazaar).
2. Coleta de amostras benignas para a contra-classe (DikeDataset).
3. Extração de características via análise estática (pescanner / PE_Analyzer).
4. Consolidação das características em um CSV rotulado.
5. Conversão do CSV para o formato LIBSVM.
6. Treinamento, comparação de kernels e validação cruzada do modelo SVM.

Etapa 1: Coleta de Amostras Maliciosas (Amadey)

Fonte: MalwareBazaar (abuse.ch), repositório público de amostras de malware rotuladas por família.

- Foi criada uma conta gratuita em abuse.ch para obter uma Auth-Key de acesso à API.
- Um script Python foi escrito para automatizar a coleta: consulta o endpoint get_siginfo filtrando pela assinatura "Amadey", baixa cada amostra pelo endpoint get_file (arquivos vêm em .zip protegido pela senha padrão "infected") e extrai o binário puro, salvo sem extensão (apenas o hash SHA256 como nome) para evitar execução acidental.
- Execução feita dentro de uma VM Kali Linux isolada de rede, dentro de um ambiente virtual Python (venv).

Resultado: 200 amostras da família Amadey coletadas com sucesso.

Etapa 2: Coleta de Amostras Benignas

Fonte: DikeDataset (github.com/iosifache/DikeDataset), dataset acadêmico com arquivos rotulados como benignos e maliciosos, usado como contra-classe para balancear o treinamento (200 benignas para 200 Amadey).

Problema encontrado

A pasta de arquivos benignos do DikeDataset mistura dois formatos: 982 arquivos PE (executáveis) e 100 arquivos OLE (documentos do Office, ex: Word/Excel), totalizando os 1082 arquivos originais da pasta. O pescanner.py (ferramenta de análise estática) só sabe interpretar estrutura PE (arquivos OLE quebravam a análise ou geravam relatórios inválidos).

Solução

Os arquivos com extensão .ole (19 deles, dentro da amostra de 200 selecionada) foram identificados e substituídos por arquivos PE novos, sorteados aleatoriamente do restante do DikeDataset, mantendo o total em 200 amostras (todas garantidamente em formato PE).

Etapa 3: Extração de Características (Análise Estática)

Ferramenta: pescanner.py (PE_Analyzer, github.com/serrastusbear/PE_Analyzer), executado em loop sobre as 400 amostras (200 Amadey + 200 benignas), gerando um relatório .txt por amostra (nomeado pelo hash SHA256).

Problemas encontrados e resoluções

Problema	Causa	Resolução
ModuleNotFoundError: exifread	Dependência do pescanner.py não instalada	pip install exifread
AttributeError: DIRECTORY_ENTRY_IMPORT	pescanner.py assume que todo PE tem tabela de imports; algumas amostras (ex: empacotadas) não têm	Reprocessamento direcionado dos arquivos afetados
Arquivos .txt vazios/incompletos	Processo anterior ainda rodando em segundo plano durante a verificação	Aguardar conclusão e reprocessar apenas os hashes pendentes
Falha ao processar arquivos .ole	Arquivos não-PE misturados na pasta de benignos (ver Etapa 2)	Substituição dos .ole por PE (Etapa 2)

Resultado: 349 de 400 amostras (87,25%) processadas com sucesso. As 51 amostras restantes foram excluídas do dataset final e documentadas como limitação da técnica de análise estática automatizada (ex: binários corrompidos, formatos atípicos, estruturas fora do padrão esperado pelo parser).

Etapa 4: Consolidação em CSV Rotulado

Um script Python (montar_csv.py) percorre todos os relatórios .txt gerados pelo pescanner, ignora automaticamente os que falharam (sem o marcador "Meta-data"), identifica o rótulo de cada amostra pela pasta de origem e extrai 10 características por meio de expressões regulares:

- file_size — tamanho do arquivo em bytes
- is_64bit — arquitetura do binário (32/64 bits)
- num_sections — número de seções do PE
- entropy_avg / entropy_max / entropy_min — entropia das seções (indício de empacotamento/ofuscação)
- date_suspicious — timestamp de compilação suspeito (ex: data zerada)
- entrypoint_suspicious — entry point fora da seção .text principal
- crc_mismatch — divergência entre CRC declarado e calculado (indício de adulteração)
- num_imported_dlls — número de DLLs únicas importadas

Resultado: dataset_amadey.csv com 349 linhas (177 Amadey / 172 benignas) e a coluna label (1 = Amadey, -1 = benigno).

Etapa 5: Conversão para o Formato LIBSVM

Ferramenta: converter_libsvm.py, do próprio repositório DejavuForensics/SVM. Alguns pontos técnicos exigiram ajuste no CSV antes da conversão:

- O script espera o separador ponto-e-vírgula (;), não vírgula — foi necessário converter o delimitador.
- A coluna sha256 (identificador de texto) foi removida antes da conversão, pois o script aplica one-hot encoding em qualquer coluna não numérica — mantê-la geraria centenas de colunas artificiais sem sentido.
- O script mapeia os dois valores da coluna de rótulo na ordem em que aparecem no arquivo (primeiro valor → +1, segundo → -1). Por isso, as linhas foram reordenadas com todas as amostras Amadey (label=1) antes das benignas, garantindo o mapeamento de sinal correto.

- A saída do script é sempre salva em uma subpasta fixa (docx_odt_oneclass_system/) dentro do repositório SVM, independentemente do caminho informado.

Resultado: dataset_amadey_SVM_format.libsvm, no formato esparso padrão do LIBSVM (índice:valor).

Etapa 6: Treinamento e Avaliação do Modelo

Primeira tentativa (kernel RBF, sem normalização)

O treinamento inicial (validação cruzada 10-fold, script svmKfold.py) apresentou overfitting severo:

Métrica	Treino	Teste
Acurácia média	100,00% ± 0,00%	70,20% ± 8,09%

Cerca de 90% das amostras de treino se tornaram vetores de suporte — sinal claro de que o modelo estava memorizando os dados em vez de generalizar.

Comparação entre kernels

Usando o script svmParameters.py (que testa Linear, Polinomial, RBF e Sigmoide com diferentes combinações de C e gamma), foi gerado um relatório comparativo:

Kernel	Acc. Treino	Acc. Teste
Polinomial (C=1, γ=1)	81,09% ± 1,22%	81,66% ± 4,47%
RBF (C=1, γ=1)	100,00% ± 0,00%	70,48% ± 3,81%
Linear	60,75% ± 7,14%	61,29% ± 7,64%
Sigmoide	50,72% ± 0,48%	50,72% ± 1,92%

O kernel Polinomial foi identificado como o mais estável, sem sinais de overfitting (treino e teste próximos).

Instabilidade numérica e normalização

Ao rodar o k-fold com o kernel Polinomial, a otimização apresentava obj = -nan e valores de rho extremamente altos (ex: 1036) — sinal de instabilidade numérica. A causa identificada foi a diferença de escala entre as features (ex: file_size na casa dos milhões vs. entropy_avg entre 0 e 8), que distorce o cálculo do kernel Polinomial.
Solução: normalização Min-Max de todas as features para o intervalo [0, 1] antes da conversão para LIBSVM, conforme recomendação oficial da documentação do LIBSVM para uso prático.

Resultado após normalização

Métrica	Antes (sem normalização)	Depois (normalizado)
Acc. Treino	81,28% ± 1,01%	96,85% ± 0,42%
Acc. Teste	81,38% ± 6,15%	95,09% ± 4,58%

Refinamento: nova busca de kernel/parâmetros pós-normalização

Como a normalização corrige a escala de todas as features, o svmParameters.py foi executado uma segunda vez sobre o dataset já normalizado, para verificar se a comparação entre kernels mudava. O resultado mostrou uma melhora generalizada — inclusive no kernel RBF, que antes apresentava overfitting severo:

Kernel	Config (C, γ)	Acc. Treino	Acc. Teste
Polinomial	C=1000, γ=1	100,00% ± 0,00%	96,27% ± 1,48%
RBF	C=1000, γ=1	99,93% ± 0,14%	96,27% ± 1,96%
Linear	C=1000, γ=0	95,41% ± 0,83%	94,25% ± 3,06%
Sigmoide	C=1, γ=1	55,01% ± 7,49%	52,17% ± 10,04%

Diferente da primeira rodada, o diagnóstico automático do relatório não indicou overfitting para nenhum dos kernels de alta acurácia (Polinomial, RBF e Linear foram classificados como "Performance Excelente"). O Polinomial (C=1000, γ=1) foi confirmado por validação cruzada 10-fold, no mesmo script svmKfold.py usado anteriormente (com o parâmetro ajustado para -t 1 -c 1000 -g 1):
Acurácia de treino: 100,00% ± 0,00% | Acurácia de teste: 96,56% ± 1,71%
Apesar dos 100% de treino, o número de vetores de suporte ficou baixo (21 a 27 de 314 amostras por fold, ~7-8%), com quase nenhum vetor de suporte "limítrofe" (nBSV próximo de 0). Isso contrasta com o cenário de overfitting real observado na primeira tentativa (277 de 314 vetores de suporte, 100 deles limítrofes) e indica que as classes ficaram bem separadas no espaço de features após a normalização, em vez de o modelo memorizar os dados.

Resultado final

Modelo final: SVM com kernel Polinomial, C=1000, γ=1, features normalizadas (Min-Max), validado por validação cruzada 10-fold. Acurácia de teste de 96,56% ± 1,71%, com gap mínimo entre treino e teste (sem overfitting).

Observações Adicionais para o Relatório

- O relatório de vazamento de dados (data leakage) do svmParameters.py não encontrou nenhuma feature com correlação suspeita (~1.0) com o rótulo, confirmando que o processo de engenharia de features não teve atalhos indevidos.
- A feature entrypoint_suspicious ficou constante (valor 1 em todas as 349 amostras), não contribuindo para a separação das classes — vale ser citada como limitação da heurística usada pelo pescanner.
- Das 400 amostras coletadas, 51 (12,75%) falharam na análise estática automatizada — principalmente por incompatibilidade de formato (OLE) e por estruturas de PE atípicas sem tabela de imports.

Referências

- MalwareBazaar (abuse.ch): bazaar.abuse.ch
- DikeDataset: github.com/iosifache/DikeDataset
- SVM (DejavuForensics): github.com/DejavuForensics/SVM
- PE_Analyzer / pescanner.py: github.com/serrastusbear/PE_Analyzer
- Apostila Blue Team (curso): capítulos 13 (análise estática/dinâmica) e 14 (classificador SVM)